

July 29, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #
    ↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as cc
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "253"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\253

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\253\Diagramme

Delete UTF8 False encoding

```

[3]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'["][a] '
#     text=re.sub(patterna, "ä", text)
#     patternu=r'["][u] '
#     text=re.sub(patternu, "ü", text)
#     patterno=r'["][o] '
#     text=re.sub(patterno, "ö", text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevocal(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)
    # Das Pattern sucht nach ` gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'`([a-zA-Z])', convert, text)

```

```

[4]: print(deletevocal("""
"""))

```

Runden signifikanter Stellen

```

[5]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran.
↳ Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt.
↳ Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geeignet sind.

    Parameter

```

```

-----
**val** : float
    Messwert

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----
**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""
einheiten_str = einheiten if einheiten is not None else ""
if errVal is not None:
    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal)))) # Die erste
    ↳signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn
    ↳1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↳Nachkommastellenposition hinzu
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'),
    ↳rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so
    ↳verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann
    ↳alle Nachkommastellen weggerundet werden
    err_round = (errVal / scale).quantize(Decimal('1'),
    ↳rounding=ROUND_CEILING) * scale

    num = f"\num{{{val_round}}({err_round})}"
    qty = f"\qty{{{val_round}}({err_round})}{{{einheiten_str}}}" if
    ↳einheiten else num

    return float(val_round), float(err_round), num, qty
else:

```

```

    val = Decimal(str(val))
    exp = int(math.floor(math.log10(float(val))))
    if round(float(val / (Decimal(10) ** exp))) < 3:           # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    val_round = (val / scale).quantize(Decimal('1'),
↳rounding=ROUND_HALF_UP) * scale
    num = f"\\num{{{val_round}}}"
    qty = f"\\qty{{{val_round}}}{einheiten_str}" if einheiten else num
    return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10^e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[6]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
↳ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
↳genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParameters** : array
        Liste aller Fehlerbehafteten Größen
    """

```

```

"""

error = 0
errProneParameters = []

function = sp.sympify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable) # Die Funktion
    ↪ wird nach der fehlerbehafteten Variable abgeleitet
    term = sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term)) # Fehler werden
    ↪ quadratisch aufsummiert
    errProneParameters.append((variable, Delta))

absolute_err = sp.simplify(sp.sqrt(error), rational = True)
relative_err = sp.simplify(sp.sqrt(error/function**2), rational = True)

if latex:
    # Prints out the Latex Code for the Functions
    print("Funktion:")
    print(sp.latex(function))
    display(Math(sp.latex(function, long_frac_ratio=2)))
    print("Absoluter Fehler:")
    print(sp.latex(absolute_err))
    display(Math(sp.latex(absolute_err, long_frac_ratio=2)))
    print("Relativer Fehler:")
    print(sp.latex(relative_err))
    display(Math(sp.latex(relative_err, long_frac_ratio=2)))
    return function, absolute_err, relative_err, errProneParameters

```

Sigma-Abweichungen

```

[7]: def sigma_abweichung(p1, err_p1, p2, err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichung von zwei Messwerten, oder einem
    ↪ Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
        ↪ fehlerbehaftet sein!")
    else:
        abweichung = Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2 + err_p2 ** 2))))

```

```

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
    ↪ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪ Nachkommastellenposition hinzu
    exp -= 1
scale = Decimal(10) ** exp
abweichung_round = (abweichung / scale).quantize(Decimal('1'),
    ↪ rounding=ROUND_CEILING) * scale
res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round), res

```

```

[8]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
    ↪ val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{xZZZx}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &
    ↪ {nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
    ↪ \\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
    ↪ err2_array):

        VAL1=round_to_sigs(val1,err1,r'[2]
        if err2!=0:

```

```

        VAL2=round_to_sigs(val2,err2,r'')[2]
    else:
        VAL2=val2

    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:
        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r'')[2]
        REL = round_to_sigs(rel, rel_delta, r'')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    # Eine saubere Zeile nur mit den Werten für LaTeX
    row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
    table_rows.append(row)

body_str = "\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{nam1}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[9]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'')[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]

```



```

        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r'')[2]
        REL = round_to_sigs(rel, rel_delta, r'')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    output = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZx}}
        \\toprule
        \\
        ↪{{nam1}}[\\unit{{einheiten}}]&{{nam2}}[\\unit{{einheiten}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{einheiten}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{percent}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """)
    print(output)

```

```

[10]: n_0=92/300
Delta_n_0=np.sqrt(92)/300

a=np.r_[np.arange(0,3.3,0.3)]
t_beta=np.r_[np.repeat(30,7),np.repeat(120,4),300]
N_beta=np.array([893,577,338,235,151,102,55,146,70,62,51,112])
n_beta=N_beta/t_beta
Delta_n_beta=np.sqrt(N_beta)/t_beta
n_0_beta=n_beta[-1]
Delta_n_0_beta=Delta_n_beta[-1]
print(round_to_sigs(n_0_beta,Delta_n_0_beta,r'\per\s')[3])

diff_n=n_beta[:-1]-n_0_beta
Delta_diff_n=np.sqrt(Delta_n_beta[:-1]**2+Delta_n_0_beta**2)

```

```

def fit_exp(x, a, b, c):
    return a+b*np.exp(-c*x)

popt, pcov=curve_fit(fit_exp,a, diff_n, p0=[[-0.3,30,1.5]],sigma=Delta_diff_n)
pcovfehler = np.sqrt(np.diag(pcov))
print(round_to_sigs(popt[0],pcovfehler[2],r'\per\s')[3])
# Parameter für +1 und -1
popt_plus = popt + pcovfehler
popt_minus = popt - pcovfehler
nullstelle=-np.log(-popt[0]/popt[1])/popt[2]
nullstelleplus=-np.log(-popt_plus[0]/popt_plus[1])/popt_plus[2]
nullstelleminus=-np.log(-popt_minus[0]/popt_minus[1])/popt_minus[2]
Delta_nullstelle=np.abs(nullstelle-nullstelleplus)
a_0,Delta_a_0,_,latexa0=round_to_sigs(nullstelle,Delta_nullstelle,r'\milli\m')
print(latexa0)
R_beta=2.6989*a_0*0.1+0.130
Delta_R_beta=2.6989*Delta_a_0*0.1
R_beta,Delta_R_beta,_,latexRbeta=round_to_sigs(R_beta,Delta_R_beta,r'\g\per\cm\squared')
print(latexRbeta)

plt.close('all')
fig, ax1 = plt.subplots(1, 1)
# fig.suptitle(r'Silbermessung')
ax1.errorbar(a,diff_n,yerr=Delta_diff_n,fmt='.',capsize= 3,elinewidth= 0.
    ↳5,label='Messwerte')# ecolor='black'
x=np.linspace(0,5,300)
ax1.plot(x, fit_exp(x,*popt),color='red',label=r'Fit $f_{A,\beta,c}$')
ax1.plot(x, fit_exp(x, *popt_plus), 'r--', label=r'Fit_
    ↳$f_{(A,\beta,c)+1\sigma}$')
ax1.plot(x, fit_exp(x, *popt_minus), 'r--', label=r'Fit_
    ↳$f_{(A,\beta,c)-1\sigma}$')

ax1.axvline(x = nullstelle, ymin = 0.001, ymax = 1e2,
    ↳linestyle='dashed',label=r'$a_0$',color = 'black')
ax1.axvline(x = nullstelleminus, ymin = 0.001, ymax = 1e2,
    ↳linestyle='dotted',label=r'$a_0-\Delta a$',color = 'black')
ax1.axvline(x = nullstelleplus, ymin = 0.001, ymax = 1e2,
    ↳linestyle='dotted',label=r'$a_0+\Delta a$',color = 'black')

ax1.set_title(r'$\beta$-Zerfall und Absorption in Aluminium')
ax1.set_xlabel(r'Absorberdicke $a$ [mm]')
ax1.set_ylabel(r'Zerfallsrate $n-n_0^{\beta}$
    ↳$[\frac{\mathrm{Counts}}{\mathrm{s}}]$')
ax1.set_yscale('log')
# ax1.set_ylim(0.001, 100)

```

```

# ax1.axhline(mean_background,linestyle=(0, (5,
↳3)),color='black',label=r'Untergrund $y_0$')

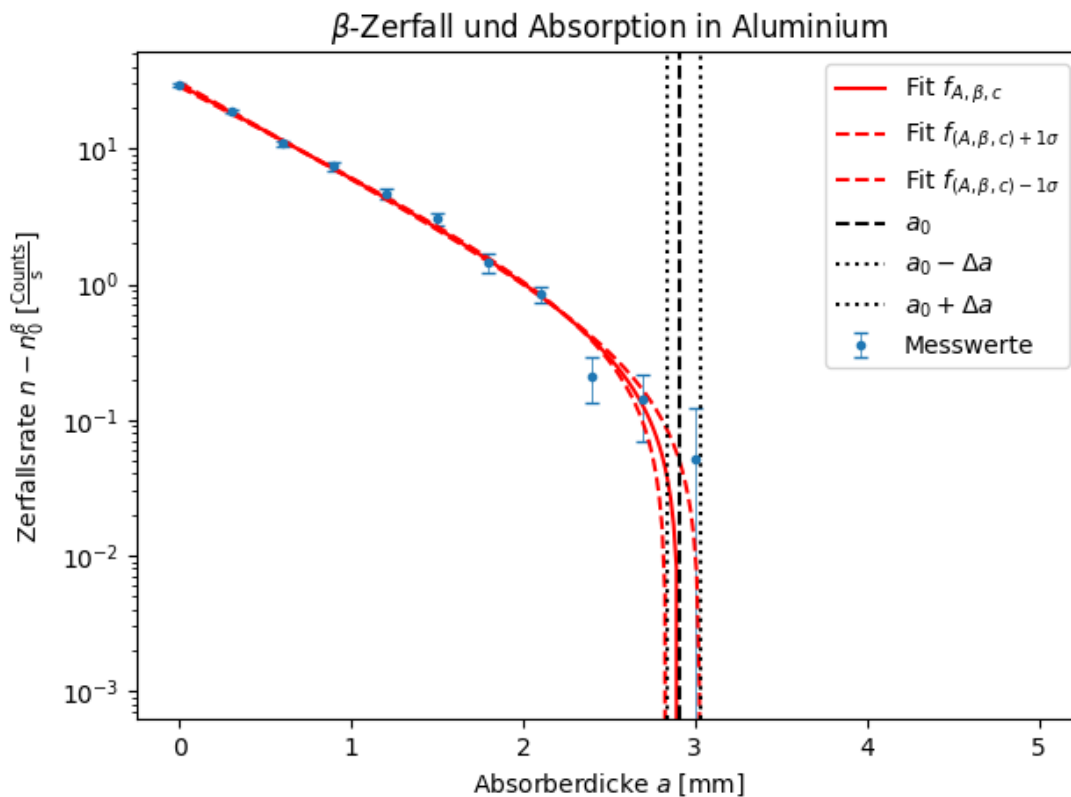
ax1.legend()
# ax1.grid()

plt.tight_layout()
plt.savefig(Ausgabe_path/'1betastrahlung_senkrecht.
↳png',format="png",bbox_inches='tight')
plt.show()

chi2_f=np.sum((fit_exp(a,*popt)-diff_n)**2/Delta_diff_n**2)
dof_f=len(diff_n)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f
print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=",round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")

```

$\text{\qty{0.37}{0.04}} \text{\per{s}}$
 $\text{\qty{-0.33}{0.06}} \text{\per{s}}$
 $\text{\qty{2.91}{0.13}} \text{\milli{m}}$
 $\text{\qty{0.92}{0.04}} \text{\g\per{cm}{squared}}$



```
chi2= 12.0
chi2_red= 1.5
Wahrscheinlichkeit= 17.0 %
```

```
[26]: n_0=92/300
Delta_n_0=np.sqrt(92)/300

a=np.r_[np.arange(0,3.3,0.3),4]
t_beta=np.r_[np.repeat(30,7),np.repeat(120,4),300]
N_beta=np.array([893,577,338,235,151,102,55,146,70,62,51,112])
n_beta=N_beta/t_beta
Delta_n_beta=np.sqrt(N_beta)/t_beta
# n_0_beta=n_beta[-1]
# Delta_n_0_beta=Delta_n_beta[-1]
# diff_n=n_beta[:-1]-n_0_beta
# Delta_diff_n=np.sqrt(Delta_n_beta[:-1]**2+Delta_n_0_beta**2)

def fit_exp(x, a, b, c):
    return a*np.exp(-b*x)+c
def fit_expp(x,a,b):
    return a*np.exp(-b*x)
popt, pcov=curve_fit(fit_exp,a, n_beta, p0=[30,1.5,-0.3],sigma=Delta_n_beta)
print(popt)
pcovfehler = np.sqrt(np.diag(pcov))
# Parameter für +1 und -1
popt_plus = popt + pcovfehler
popt_minus = popt - pcovfehler

poptlinear,pcovlinear=curve_fit(fit_expp,a[:-3],n_beta[:-3],p0=[30,1.
↪5],sigma=Delta_n_beta[:-3])
a_0=-np.log(popt[2]/poptlinear[0])/poptlinear[1]
a_0_plus=-np.log(popt_plus[2]/poptlinear[0])/poptlinear[1]
a_0_minus=-np.log(popt_minus[2]/poptlinear[0])/poptlinear[1]
a_0,Delta_a_0,_,latexa0=round_to_sigs(a_0,np.max([np.abs(a_0-a_0_plus),np.
↪abs(a_0-a_0_minus)]),r'\mm')
print("C"+latexa0)
R_beta=2.6989*a_0*1e-1+0.130
Delta_R_beta=2.6989*Delta_a_0*1e-1
R_beta,Delta_R_beta,_,latexRbeta=round_to_sigs(R_beta,Delta_R_beta,r'\g\per\cm\squared')
print("C"+latexRbeta)

# nullstelle=-np.log(-popt[0]/popt[1])/popt[2]
# nullstelleplus=-np.log(-popt_plus[0]/popt_plus[1])/popt_plus[2]
# nullstelleminus=-np.log(-popt_minus[0]/popt_minus[1])/popt_minus[2]
# Delta_nullstelle=np.abs(nullstelle-nullstelleplus)
```

```

# a_0,Delta_a_0,_,latexa0=round_to_sigs(nullstelle,Delta_nullstelle,r'\milli\m')
# print(latexa0)
# R_beta=2.6989*a_0*10+0.130
# Delta_R_beta=2.6989*Delta_a_0*10
#_
    ↪ R_beta,Delta_R_beta,_,latexRbeta=round_to_sigs(R_beta,Delta_R_beta,r'\g\per\cm\squared')
# print(latexRbeta)

plt.close('all')
fig, ax1 = plt.subplots(1, 1)
# fig.suptitle(r'Silbermessung')
ax1.errorbar(a,n_beta,yerr=Delta_n_beta,fmt='.',capsize= 3,elinewidth= 0.
    ↪ 5,label='Messwerte')# ecolor='black'
x=np.linspace(0,5,300)
ax1.plot(x, fit_exp(x,*popt),color='red',label=r'Fit  $f_{A,\beta,C}$ ')
ax1.plot(x, fit_exp(x, *popt_plus), 'r--', label=r'Fit_
    ↪  $f_{(A,\beta,C)+1\sigma}$ ')
ax1.plot(x, fit_exp(x, *popt_minus), 'r--', label=r'Fit_
    ↪  $f_{(A,\beta,C)-1\sigma}$ ')
xminus=np.linspace(0,a_0_minus,10)
ax1.
    ↪ plot(xminus,fit_expp(xminus,*poptlinear),linestyle='dashed',label=r' $f_{A,\beta}$ ',color_
    ↪ 'black')

ax1.hlines(popt[2]+pcovfehler[2],xmin=a_0_plus,xmax=5,linewidth=0.7,color =_
    ↪ 'black')
ax1.hlines(popt[2],xmin=a_0,xmax=5,linestyle='dotted',label=r' $C\pm\Delta C$ ',color = 'black')
ax1.hlines(popt[2]-pcovfehler[2],xmin=a_0_minus,xmax=5,linewidth=0.7,color =_
    ↪ 'black')
ax1.
    ↪ scatter([a_0_plus,a_0,a_0_minus],popt[2]+[+pcovfehler[2],0,-pcovfehler[2]],marker='|',s=100
    ↪  $\Delta a^C_0$ )

a_0=-np.log(n_0_beta/poptlinear[0])/poptlinear[1]
a_0_plus=-np.log((n_0_beta+Delta_n_0_beta)/poptlinear[0])/poptlinear[1]
a_0_minus=-np.log((n_0_beta-Delta_n_0_beta)/poptlinear[0])/poptlinear[1]
a_0,Delta_a_0,_,latexa0beta=round_to_sigs(a_0,np.max([np.abs(a_0-a_0_plus),np.
    ↪ abs(a_0-a_0_minus)]),r'\mm')
print("a_n_0beta"+latexa0beta)
R_beta=2.6989*a_0*1e-1+0.130
Delta_R_beta=2.6989*Delta_a_0*1e-1
R_beta,Delta_R_beta,_,latexRbeta=round_to_sigs(R_beta,Delta_R_beta,r'\g\per\cm\squared')
print("n_0beta:"+latexRbeta)

```

```

ax1.hlines(n_0_beta+Delta_n_0_beta,xmin=a_0_plus,xmax=4,linewidth=0.7,color = 'C0')
ax1.
    ↪hlines(n_0_beta,xmin=a_0,xmax=4,linestyle='dotted',label=r'$n_0^{\beta}\pm\Delta n_0^{\beta}$',color = 'C0')
ax1.hlines(n_0_beta-Delta_n_0_beta,xmin=a_0_minus,xmax=4,linewidth=0.7,color = 'C0')
# ax1.axvline(x = nullstelle, ymin = 0.001, ymax = 1e2,
    ↪linestyle='dashed',label=r'$a_0$',color = 'black')
# ax1.axvline(x = nullstelleminus, ymin = 0.001, ymax = 1e2,
    ↪linestyle='dotted',label=r'$a_0-\Delta a$',color = 'black')
# ax1.axvline(x = nullstelleplus, ymin = 0.001, ymax = 1e2,
    ↪linestyle='dotted',label=r'$a_0+\Delta a$',color = 'black')

ax1.set_title(r'$\beta$-Zerfall und Absorption in Aluminium')
ax1.set_xlabel(r'Absorberdicke $a$ [mm]')
ax1.set_ylabel(r'Zerfallsrate $n_0^{\beta}$
    ↪[$\frac{\mathrm{Counts}}{\mathrm{s}}$'])
ax1.set_yscale('log')
# ax1.set_ylim(0.001, 100)
# ax1.axhline(mean_background,linestyle=(0, (5,
    ↪3)),color='black',label=r'Untergrund $y_0$')

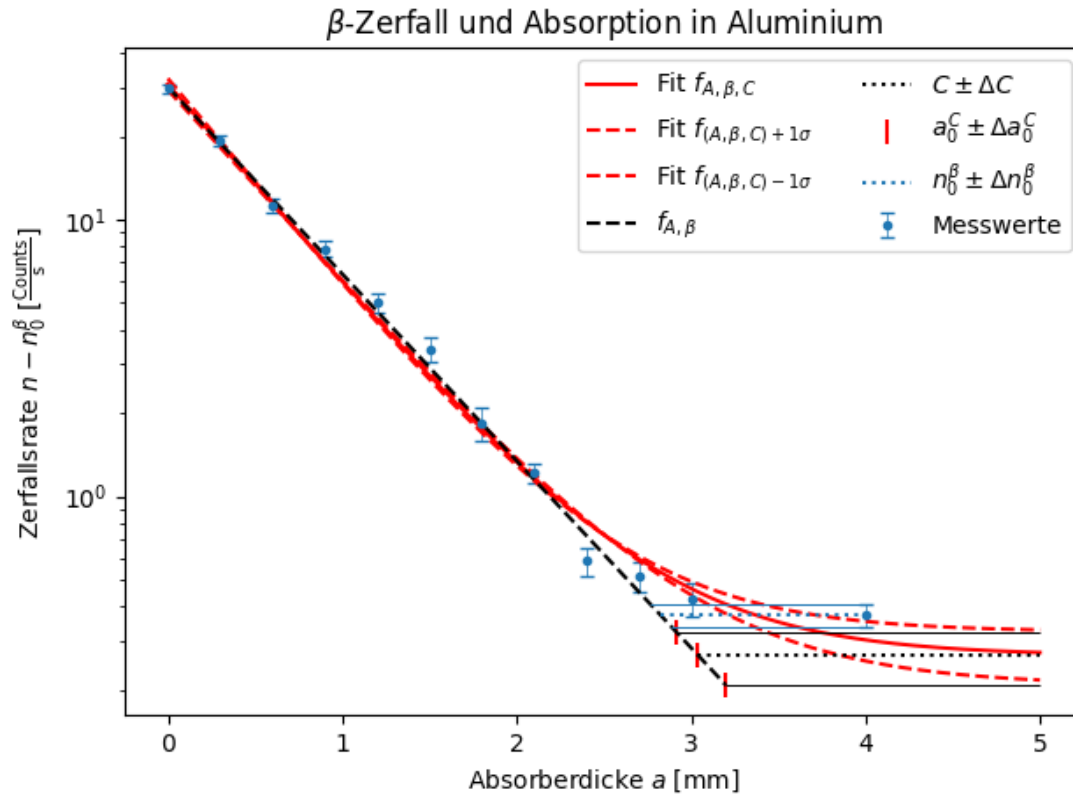
ax1.legend(ncols=2)
# ax1.grid()

plt.tight_layout()
plt.savefig(Ausgabe_path/'1betastrahlung_neu.
    ↪png',format="png",bbox_inches='tight')
plt.show()

chi2_f=np.sum((fit_exp(a,*popt)-n_beta)**2/Delta_n_beta**2)
dof_f=len(n_beta)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f
print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=",round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")

[30.64553548  1.68024071  0.2657087 ]
C\qty{3.04(0.16)}{\mm}
C\qty{0.95(0.05)}{\g\per\cm\squared}
a_n_0beta\qty{2.82(0.07)}{\mm}
n_0beta:\qty{0.891(0.019)}{\g\per\cm\squared}

```



```
chi2= 30.0
chi2_red= 3.0
Wahrscheinlichkeit= 0.0 %
```

```
[12]: compare_table(r'E^{\mathrm{exp}}_{\beta}',r'E^{\mathrm{lit}}_{\beta}',r'\mega\electron\volt',2.
    ↪01,0.15,2.274,0)
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $E^{\mathrm{exp}}_{\beta}$  und  $E^{\mathrm{lit}}_{\beta}$ }
\begin{tabularx}{\textwidth}{ZZZZx}
\toprule
E^{\mathrm{exp}}_{\beta}[\unit{\mega\electron\volt}]&E^{\mathrm{lit}}_{\beta}[\unit{\mega\electron\volt}]&\text{abs. Abw.}[\unit{\mega\electron\volt}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{num}{2.01(0.15)}&\text{num}{2.274}&\text{num}{0.26(0.15)}&\text{num}{13(8)}&\text{num}{1.8}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_E^{\mathrm{exp}}_{\beta}}
\end{table}
```



```

[31]: n_0=92/300
Delta_n_0=np.sqrt(92)/300

a=np.r_[np.arange(0,5.5,0.5)]
t_gamma=np.repeat(60,len(a))

N_gamma=np.array([629,448,313,215,178,158,108,87,56,53,49])
n_gamma=N_gamma/t_gamma
Delta_n_gamma=np.sqrt(N_gamma)/t_gamma
diff_n=n_gamma-n_0
Delta_diff_n=np.sqrt(Delta_n_gamma**2+Delta_n_0**2)

def fit_exp(x, a, c):
    return a*np.exp(-c*x)

popt, pcov=curve_fit(fit_exp,a, diff_n, p0=[[0.5,1]],sigma=Delta_diff_n)
pcovfehler = np.sqrt(np.diag(pcov))
# Parameter für +1 und -1
murho=popt[1]/11.342
Delta_murho=np.sqrt(pcov[1][1])/11.342
murho,Delta_murho,_,latexmurho=round_to_sigs(murho,Delta_murho,r'\cm\squared\g')
print(latexmurho)

mu,Delta_mu,_,latexmu=round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\per\cm')
print(latexmu)

plt.close('all')
fig, ax1 = plt.subplots(1, 1)
# fig.suptitle(r'Silbermessung')
ax1.errorbar(a,diff_n,yerr=Delta_diff_n,fmt='.',capsize= 3,elinewidth= 0.
    ↳5,label='Messwerte')# ecolore='black'
x=np.linspace(0,5,300)
ax1.plot(x, fit_exp(x,*popt),color='red',label=r'Lambert-Beer Fit')

ax1.set_title(r'\gamma-Zerfall und Absorption in Blei')
ax1.set_xlabel(r'Absorberdicke $a$ [cm]')
ax1.set_ylabel(r'Zerfallsrate $n-n_0$ [$\frac{\mathrm{Counts}}{\mathrm{s}}$]')
ax1.set_yscale('log')
# ax1.axhline(mean_background,linestyle=(0, (5,
    ↳3)),color='black',label=r'Untergrund $y_0$')

ax1.legend()
# ax1.grid()

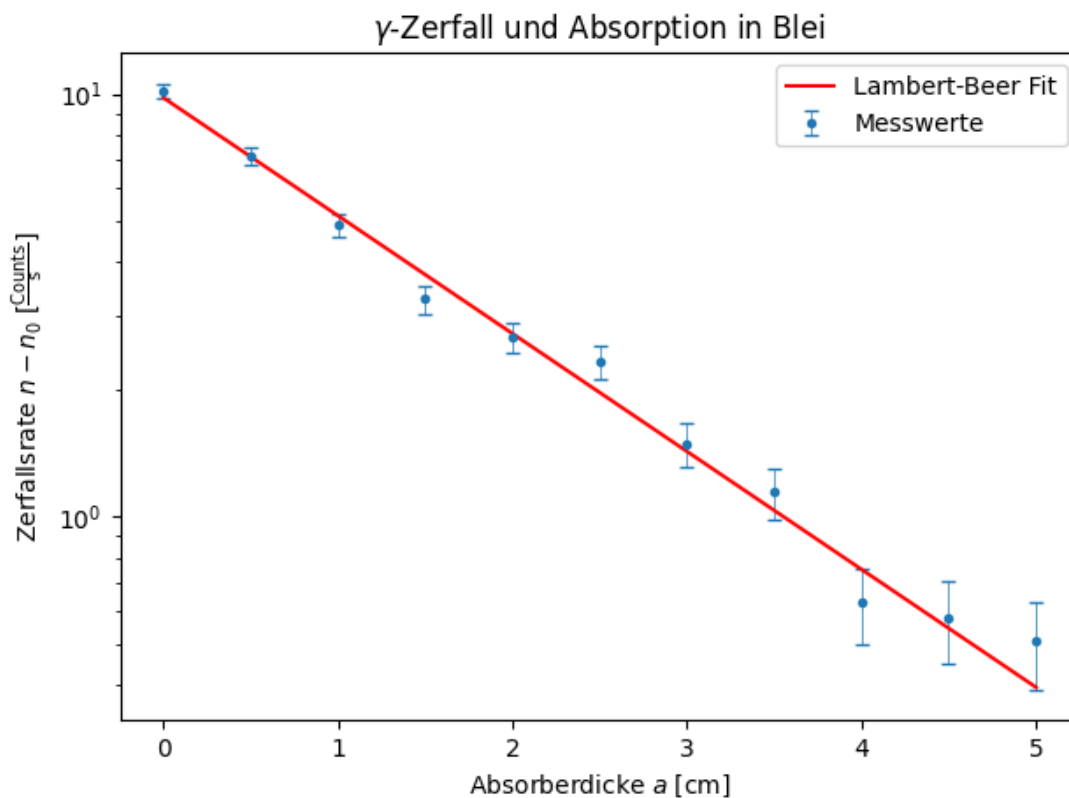
plt.tight_layout()
plt.savefig(Ausgabe_path/'2_gammastrahlung.
    ↳png',format="png",bbox_inches='tight')

```

```
plt.show()

chi2_f=np.sum((fit_exp(a,*popt)-diff_n)**2/Delta_diff_n**2)
dof_f=len(diff_n)-3 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f
print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=",round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")
```

$\chi^2 = 10.0566(0.0020)$ cm^2
 $\chi^2_{\text{red}} = 0.642(0.023)$ per cm



```
chi2= 10.0
chi2_red= 1.3
Wahrscheinlichkeit= 23.0 %
```

```
[14]: compare_table_array(r'E^{\text{exp}}_\gamma',r'E^{\text{lit}}_\gamma',r'\mega\electronvolt',[1
↪30,1.30],[0.1,0.1],[1.1730,1.333],[0,0])
```

```
\begin{table}[htbp]
\centering
```

```

\caption{Vergleich von  $E^{\text{exp}}_{\gamma}$  und  $E^{\text{lit}}_{\gamma}$ }
\begin{tabularx}{\textwidth}{xZZZZx}
\toprule
Messreihe &  $E^{\text{exp}}_{\gamma}$  [\unit{mega\electronvolt}] &
 $E^{\text{lit}}_{\gamma}$  [\unit{mega\electronvolt}] &
\text{abs.Abw.} [\unit{mega\electronvolt}] & \text{proz.Abw.} [\unit{percent}] &
S[\sigma] \\
\midrule
& \num{1.30(0.10)} & 1.173 & \num{0.13(0.10)} & \num{10(8)} & \num{1.3} \\
\\
& \num{1.30(0.10)} & 1.333 & \num{0.03(0.10)} & \num{3(8)} & \num{0.4} \\
\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_E^{\text{exp}}_{\gamma}}
\end{table}

```

```

[15]: A_0=3700*1000
t=514771200
T_12=5.27*365.2425*24*3600
A=A_0*2**(-t/T_12)
print(A)

```

432923.4112917773

```

[16]: N=np.array([7147,2048,558])
t=60
n=N/t-n_0
Delta_n=np.sqrt((np.sqrt(N)/t)**2+Delta_n_0**2)
d=np.array([5,10,20])*1e-2 # cm in m
Delta_d=np.sqrt(2)*0.1*1e-2 # cm in m
epsilon=0.04
r=0.7*1e-2 # cm in m
l=4*1e-2 # cm in m
A=4*n*d**2/(epsilon*r**2)/1000 # kilo bq
Delta_A=A*np.sqrt(4*Delta_d**2/d**2 + Delta_n**2/n**2)
A,Delta_A,_,latexA=v_round(A,Delta_A,r'\kilo\becquerel')
# print(latexA[i]) for i in range(3):
print("A")
for i in range(3):
    print(latexA[i])

A_korr=4*n*(d+l/2)**2/(epsilon*r**2)/1000 # kilo bq
Delta_A_korr=A_korr*np.sqrt(16*Delta_d**2/(2*d + l)**2 + Delta_n**2/n**2)
A_korr,Delta_A_korr,_,latexA_korr=v_round(A_korr,Delta_A_korr,r'\kilo\becquerel')
# print(latexA[i]) for i in range(3):
print("A_korr")
for i in range(3):
    print(latexA_korr[i])

```

```

k_1=1+1/d+1**2/(4*d**2)
Delta_k_1=np.sqrt(Delta_d**2*1**2*(2*d + 1)**2/d**6)/2
k_1,Delta_k_1,_,latexk1=v_round(k_1,Delta_k_1,r'')
print("k_1")
for i in range(3):
    print(latexk1[i])

A_k_korr=A*k_1
Delta_A_k_korr=A_k_korr*np.sqrt(Delta_k_1**2/k_1**2 + Delta_A**2/A**2)
A_k_korr,Delta_A_k_korr,_,latexA_k_korr=v_round(A_k_korr,Delta_A_k_korr,r'\kilo\becquerel')
print("A_k_korr")
for i in range(3):
    print(latexA_k_korr[i])

```

```

A
\qty{610(40)}{\kilo\becquerel}
\qty{690(25)}{\kilo\becquerel}
\qty{730(40)}{\kilo\becquerel}
Akorr
\qty{1190(60)}{\kilo\becquerel}
\qty{990(40)}{\kilo\becquerel}
\qty{890(50)}{\kilo\becquerel}
k_1
\num{1.96(0.04)}
\num{1.440(0.007)}
\num{1.2100(0.0016)}
A_k_korr
\qty{1200(90)}{\kilo\becquerel}
\qty{990(40)}{\kilo\becquerel}
\qty{880(50)}{\kilo\becquerel}

```

```

[17]: compare_table_array(r'A_{\text{korr}}^{\text{exp}}',r'A_{\text{korr}}^{\text{lit}}',r'\kilo\becquerel')
      ↪ repeat(433,3),np.repeat(0,3))

```

```

\begin{table}[htbp]
\centering
\caption{Vergleich von  $A_{\text{korr}}^{\text{exp}}$  und  $A_{\text{korr}}^{\text{lit}}$ }
\begin{tabularx}{\textwidth}{xZZZZx}
\toprule
Messreihe &  $A_{\text{korr}}^{\text{exp}}$  [ $\text{kilo\becquerel}$ ] & & & &
 $A_{\text{korr}}^{\text{lit}}$  [ $\text{kilo\becquerel}$ ] & & & &
\text{abs. Abw.} [ $\text{kilo\becquerel}$ ] & \text{proz. Abw.} [ $\text{percent}$ ] & &
S[\sigma] \\
\midrule
& \num{1200(90)} & 433 & \num{770(90)} & \num{64(9)} & \num{9} & \backslash\backslash
& \num{990(40)} & 433 & \num{560(40)} & \num{56(5)} & \num{14} & \backslash\backslash
& \num{880(50)} & 433 & \num{450(50)} & \num{51(7)} & \num{9} & \backslash\backslash

```

```

\bottomrule
\end{tabularx}
\label{table:Vergleich_A_{\text{korr}}^{\text{exp}}}
\end{table}

```

```
[18]: gff("e**(murho*rho_ab*x)", 'murho')
```

Funktion:

$$e^{\text{murho} \cdot \rho_{ab} \cdot x}$$

$$e^{\text{murho} \rho_{ab} x}$$

Absoluter Fehler:

$$\sqrt{\Delta_{\text{murho}}^2 e^{2 \text{murho} \cdot \rho_{ab} \cdot x} \rho_{ab}^2 x^2 \log(e)^2}$$

$$\sqrt{\Delta_{\text{murho}}^2 e^{2 \text{murho} \rho_{ab} x} \rho_{ab}^2 x^2 \log(e)^2}$$

$$\sqrt{\Delta_{\text{murho}}^2 e^{2 \text{murho} \rho_{ab} x} \rho_{ab}^2 x^2 \log(e)^2}$$

Relativer Fehler:

$$\sqrt{\Delta_{\text{murho}}^2 \rho_{ab}^2 x^2 \log(e)^2}$$

$$\sqrt{\Delta_{\text{murho}}^2 \rho_{ab}^2 x^2 \log(e)^2}$$

```
[18]: (e**(murho*rho_ab*x),
      sqrt(Delta_murho**2*e**(2*murho*rho_ab*x)*rho_ab**2*x**2*log(e)**2),
      sqrt(Delta_murho**2*rho_ab**2*x**2*log(e)**2),
      [(murho, Delta_murho)])
```

```
[19]: rho_ab=7.9
x=1.4*0.1
A_ab=A_k_korr*np.exp(murho*rho_ab*x)
print(round_to_sigs(np.exp(murho*rho_ab*x),Delta_murho*rho_ab*x*np.
    ↳exp(murho*rho_ab*x))[3])
Delta_A_ab=A_ab*np.sqrt(Delta_murho**2*rho_ab**2*x**2+ Delta_A_k_korr**2/
    ↳A_k_korr**2)
A_ab,Delta_A_ab,_,latexAab=v_round(A_ab,Delta_A_ab,r'\kilo\becquerel')
print("A_ab")
for i in range(3):
    print(latexAab[i])
```

$$\text{num}\{1.0646(0.0024)\}$$

$$A_{ab}$$

$$\text{qty}\{1280(100)\}\{\text{kilo}\text{becquerel}\}$$

$$\text{qty}\{1050(50)\}\{\text{kilo}\text{becquerel}\}$$

$$\text{qty}\{940(60)\}\{\text{kilo}\text{becquerel}\}$$

```
[20]: compare_table_array(r'A_{\text{exp}}',r'A_{\text{lit}}',r'\kilo\becquerel',[1280,1050,940],[100,
    ↳repeat(433,3),np.repeat(0,3))
```

```
\begin{table}[htbp]
```



```

Delta_x_max_2_minus=(np.tan((Delta_n_max_2_minus-popt[3])/popt[0])+popt[2])/
    ↪popt[1]
Delta_x_max_2=np.max([np.abs(x_max_2-Delta_x_max_2_plus),np.
    ↪abs(x_max_2-Delta_x_max_2_minus)])
p_1,Delta_p_1,_,latexp1=round_to_sigs(x_max_2,Delta_x_max_2,r'\milli\bar')
print(latexp1)

ax1.
    ↪hlines(n_max_2,xmin=0,xmax=x_max_2,linestyle='dashed',color='red',label=r'$\frac{n_{\max}}{2}$')
ax1.
    ↪vlines(x_max_2,ymin=0,ymax=n_max_2,linestyle='dashed',color='black',label=r'$p(\frac{n_{\max}}{2})$')
ax1.scatter(x_max_2,n_max_2,color='red',marker='x')

ax1.hlines(Delta_n_max_2_plus,xmin=0,xmax=Delta_x_max_2_plus,linewidth=0.
    ↪6,color='red')
ax1.hlines(Delta_n_max_2_minus,xmin=0,xmax=Delta_x_max_2_minus,linewidth=0.
    ↪6,color='red')
ax1.vlines(Delta_x_max_2_plus,ymin=0,ymax=Delta_n_max_2_plus,linewidth=0.
    ↪6,color='black')
ax1.vlines(Delta_x_max_2_minus,ymin=0,ymax=Delta_n_max_2_minus,linewidth=0.
    ↪6,color='black')

ax1.legend()
# ax1.grid()

plt.tight_layout()
plt.savefig(Ausgabe_path/'4_gammastrahlung.
    ↪png',format="png",bbox_inches='tight')
plt.show()

```

\qty{359.7(0.3)}{\milli\bar}


```
[24]: compare_table(r'E_\alpha^\text{exp}',r'E_\alpha^\text{lit}',r'\mega\electronvolt',5.  
↪50,0.01,5.48,0)
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $E_\alpha^\text{exp}$  und  $E_\alpha^\text{lit}$ }
\begin{tabularx}{\textwidth}{ZZZZx}
\toprule
E_\alpha^\text{exp}[\unit{\mega\electronvolt}]&E_\alpha^\text{lit}[\unit{\mega\electronvolt}]&\text{abs. Abw.}[\unit{\mega\electronvolt}]&\text{proz. Abw.}[\unit{\percent}]&S[\sigma]\\\midrule
\text{num}{5.500(0.010)}&\text{num}{5.48}&\text{num}{0.020(0.010)}&\text{num}{0.36(0.19)}&\text{num}{2.0}\\\bottomrule
\end{tabularx}
\label{table:Vergleich_E_\alpha^\text{exp}}
\end{table}
```